Programación de aplicaciones de red



GP21

Programación con el interfaz sockets

En esta práctica se va a trabajar sobre la competencia de programación de aplicaciones, en concreto programando una aplicación servidora empleando la interfaz que nos proporcionan los sockets.

Evaluable en test al final de la sesión

Programación con el interfaz Socket

En esta práctica se revisará el funcionamiento de un servicio de ECO, en donde el servidor devolverá la misma cadena que le ha enviado un cliente. La tarea propuesta para la práctica consiste en terminar el código existente (puede disponer de una copia en Aula Virtual) correspondiente al servidor conectivo. No hará falta implementar el cliente dado que nos va a valer con un cliente `telnet`.

Ruta rápida

1. Abra **dos terminales**: el Terminal 1 será para el servidor y el Terminal 2 para el cliente. En ambos, sitúese en la carpeta donde descargó el archivo (normalmente `cd ~/Descargas`).
2. Complete las `XXXXXXX` del archivo `servidor_a_completar_Python3.py` ayudándose del diagrama de sockets TCP (Figura 1).
3. **Terminal 1:** `python3 servidor_a_completar_Python3.py`. Debe aparecer "Socket escuchando...".
4. Abra Wireshark, capture en la interfaz `lo` (loopback) y aplique el filtro `tcp.port == 8888`.
5. **Terminal 2:** `telnet 127.0.0.1 8888`. Escriba un texto y pulse Intro: debe recibir el mismo texto (eco).
6. Para cerrar el cliente, escriba `EXIT`. Para detener el servidor, pulse `Ctrl+C` en el Terminal 1.
7. Responda a las preguntas del final. En Wireshark, "clic derecho → Seguir → Flujo TCP" muestra la conversación completa.

El alumno deberá incorporar las sentencias que sean necesarias sustituyendo las ***"XXXXXXX"*** que aparecen en el código para que pueda haber una comunicación entre el cliente (empleando el cliente telnet) y el servidor realizado. Como ayuda se remite a la ayuda oficial de Python[1] y el diagrama de sockets TCP de la Figura 1.

Las comunicaciones se realizarán mediante el protocolo TCP, sobre un puerto a determinar por el alumno. Para evitar tener problemas en la escucha del servidor, el puerto tendrá siempre un valor superior a 5000. Para la conexión desde el cliente, se utilizarán sockets, que en Python se manejan a través de la librería `socket`[2].

Código del servidor

A continuación, se tiene la parte del código del servidor de ECO.

```
import socket
import sys

HOST = ''  # Nombre o direccion de la maquina. Puede estar vacio en el servidor
PORT = 8888  # Puerto de escucha del servidor. PORT >= 5000
BUFFER = 1024  # Tamano del buffer de recepcion

# Primero se crea el socket
s = socket.XXXXXXX(socket.AF_INET, socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

print('Socket creado...')

# Enlaza el socket con la direccion y puerto local
try:
```

[1] https://docs.python.org/3/
[2] https://docs.python.org/3/library/socket.html

```
    s.bind((XXXXXXX, XXXXXXX))

# Captura excepciones. Si existe, un error sale del programa
except socket.error as msg:
    print('Fallo en BIND. Cod. Error: {} Mensaje: {}'.format(msg.errno, msg.strerror))
    sys.exit()

print('Socket Enlazado...')

# Comienza la escucha en el socket
s.listen(10)
print('Socket escuchando...')

while True:
    try:
        # Espera a que lleguen clientes. Se aceptan conexiones.
        cliente, addr = s.XXXXXXX()
        print('Conectado con {}:{}'.format(addr[0], addr[1]))

        while True:
            try:
                # Recepcion de datos. Se reciben datos del cliente.
                datos = cliente.XXXXXXX(XXXXXXX)
            except ConnectionResetError:
                # El cliente ha cortado la conexion de forma abrupta (RST)
                datos = b''

            # Si recv() devuelve b'' (vacio), el cliente ha cerrado la conexion
            if not datos:
                print('Cliente {}:{} desconectado'.format(addr[0], addr[1]))
                cliente.close()
                break

            # Se envia al cliente lo mismo que se ha recibido (eco).
            cliente.XXXXXXX(XXXXXXX)
            # Se imprime pasando un byte string a string.
            # errors='replace' evita que el servidor falle si llegan bytes que no
            # son texto (p. ej., codigos de control de telnet al pulsar Ctrl+C)
            print('Recibido: {}'.format(datos.decode('utf-8', errors='replace')))

            # Se dejara de trabajar con el cliente si se recibe EXIT.
            # Se compara con un byte string (b'EXIT'): los dos tipos deben ser iguales
            if datos[0:4] == b'EXIT':
                cliente.close()
                break

    except KeyboardInterrupt:
        print('\nCerrando socket y apagando servidor...')
        break

s.close()
```

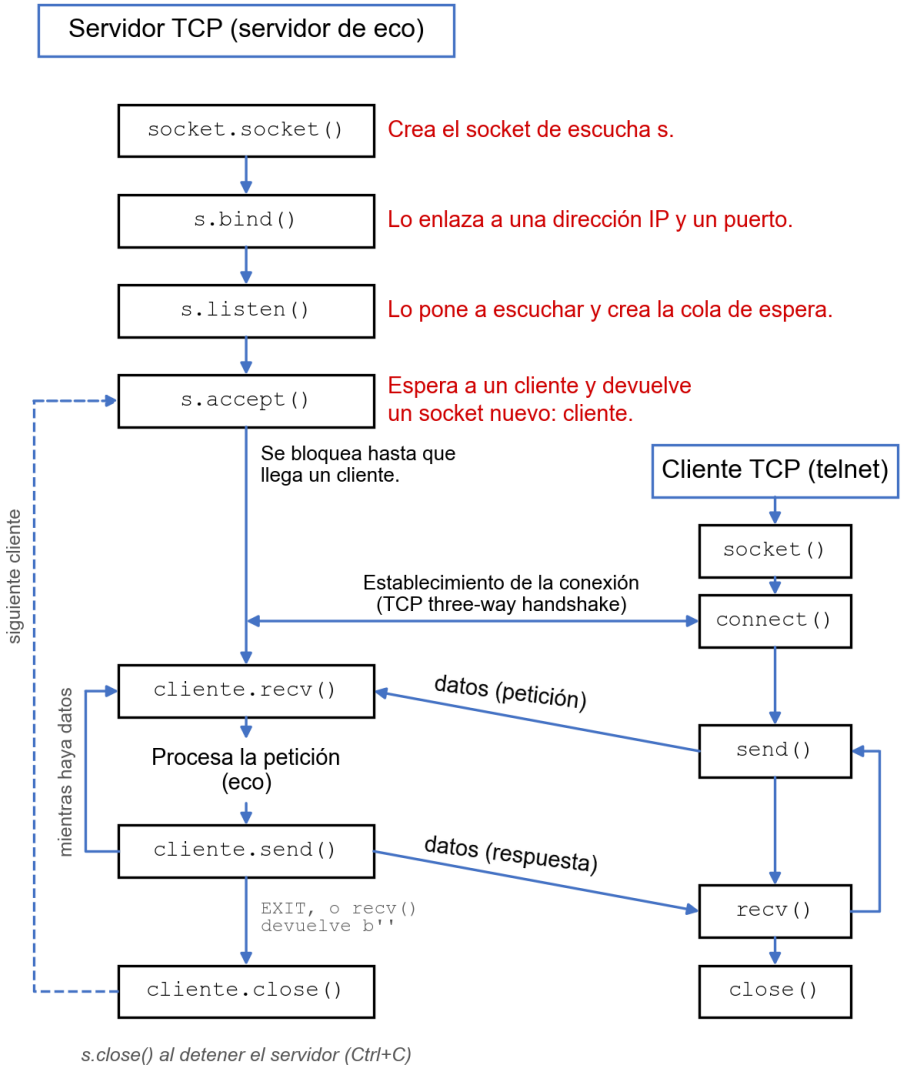


Figura 1. *Llamadas de sockets TCP del servidor de eco y del cliente (telnet), con los nombres que se usan en el código.*

El código anterior se puede encontrar en el archivo `servidor_a_completar_Python3.py`. Se pide al alumno modificar el código cambiando las cadenas XXXXXXX por código correcto que permita conectar con el servidor.

A la hora de incorporar el código hay que tener en cuenta especialmente:

- Las llaves en Python se implementan a través de la **indentación con 4 espacios por nivel. No mezcle espacios y tabuladores.**
- **Trabaje sobre el archivo .py**, no copie el código desde este documento: al copiar desde un documento se pueden alterar la indentación o las comillas.

Ejecución y captura con Wireshark

Para ejecutar el código, se debe utilizar el intérprete de Python **en un terminal**:

```
$ python3 <nombre_del_programa.py>
```

Por ejemplo, para el código del servidor, debidamente modificado (desde la carpeta donde está el archivo):

```
$ python3 servidor_a_completar_Python3.py
```

Ejecute el programa Wireshark y capture en el interfaz de loopback (lo) que permite las comunicaciones internas dentro de su equipo. A medida que vaya ejecutando los comandos que se detallarán a continuación, compruebe en Wireshark que se han intercambiado datos entre diferentes aplicaciones cliente y servidor.

Para conectar un cliente telnet con el servidor, se debe utilizar **en otro terminal** la orden telnet, seguida de la dirección (127.0.0.1 si estamos ejecutando cliente y servidor en la misma máquina) y del puerto del servidor (el configurado en el código del servidor):

```
$ telnet 127.0.0.1 8888
```

Para desconectar el *cliente* del servidor se debe enviar la palabra "EXIT" **(en mayúsculas)**. El *servidor* se puede detener utilizando la combinación de teclas **Ctrl+C.**

Cuestiones

Tras la realización del código y verificar que funciona (esto es, es posible conectar con el servidor a través de `telnet` y se pueden enviar y recibir mensajes), se pide contestar a las siguientes preguntas:

1. Al lograr una conexión TCP, el servidor imprime una línea que comienza por "Conectado con" ¿Qué datos aparecen a continuación?

Responda en el examen de Blackboard

2. ¿Puede encontrar estos datos en las tramas capturadas en Wireshark?

Responda en el examen de Blackboard

3. Si cerramos la conexión y volvemos a conectarnos, ¿varían los puertos origen y destino?

Responda en el examen de Blackboard

Localhost y 127.0.0.1 son alias que representan a la misma máquina en la que se ejecuta un software. Por esa razón, para conectar un cliente al servidor que se ejecuta en el mismo puesto de laboratorio usamos el comando anterior. Pero también podríamos realizar una conexión entre distintos puestos del laboratorio.

4. ¿Qué comando debe ejecutar para poder conectar un cliente telnet local con el servidor de otro puesto de laboratorio?

Responda en el examen de Blackboard

5. Una vez realizada la conexión, observe si puede capturar los mensajes salientes en Wireshark. En caso negativo, indique en qué otra interfaz debe capturar tráfico para poder observar estos mensajes, y por qué.

Responda en el examen de Blackboard

6. Pruebe a conectar varios clientes al mismo servidor e intercambiar mensajes desde ambos a la vez. ¿Cuál es el comportamiento del servidor?

Responda en el examen de Blackboard

7. Al cerrar uno de los clientes, ¿qué observa?

Responda en el examen de Blackboard

8. ¿A qué se debe el comportamiento observado en las dos preguntas anteriores?

Responda en el examen de Blackboard

Tras cerrar el servidor y reiniciarlo, pruebe a conectarse al mismo desde un navegador web (usando la dirección y el puerto del servidor, *p.ej. http://127.0.0.1:8888*). Puede que tenga que cerrar la conexión manualmente para visualizar el resultado.

9. ¿Qué ocurre en el servidor? ¿Qué se ve en el navegador?

Responda en el examen de Blackboard